

Parallelization of One Dimensional First Fit Decreasing Algorithm

Eriny Wessa

Faculty of Computer Science

October University for Modern Sciences and Arts (MSA)

Cairo, Egypt

eriny.yousrey@msa.edu.eg

Ayman Atia

HCI-LAB, Faculty of Computers and

Artificial Intelligence

Helwan University

Faculty of Computer Science

October University for

Modern Sciences and Arts (MSA)

Giza Egypt

Email: aezzatz@msa.edu.eg

Abstract—Bin packing is an optimization problem defined as placing different sized objects into similar containers or bins to minimize the number of used bins. This problem has different variations based on the dimensions of the bins, placement limitations, and priority. This paper focuses on one-dimensional bin packing. Two algorithms are explored in this paper, which are First Fit, First Fit Decreasing. The contribution of the paper is to explore the effect of parallelization on the First Fit Decreasing algorithm regarding the processing time and the utilization rate. Furthermore, the effect of using different numbers of concurrent workers on the problem is also explored. We proved that just by parallelizing the sorting in the First Fit Decreasing algorithm, we can decrease the computation time by 4.73% without affecting the utilization rate.

Index Terms—Bin Packing, Parallel processing, Energy efficiency.

I. INTRODUCTION

The one-dimensional bin packing problem is an NP-hard optimization problem [8] [4] [12] defined as placing different sized objects into similar containers or bins to minimize the number of used bins. In other words, given a list of N objects where each object has a different size or weight, we are asked to pack them into identical bins with a fixed capacity while using the least amount of bins possible. Furthermore, numerous variants of the problem exist in practice based on the dimensions of the bins, placement limitations, and priority [14] [13].

One dimensional bin packing has various applications that effect Energy and resource consumption. For example, bin packing can be used in memory and resource allocation. Moreover bin packing is also used in scheduling tasks on parallel processors to improve performance [2] [6] [11]. Furthermore, bin packing is also used in line balancing where all workstations have a predefined cycle time, the task is to assign a list of operations to a number of workstations while minimizing the number of workstations used [5] allowing for more efficient energy consumption.

The First Fit (FF), and the First Fit Decreasing (FFD) algorithms are variants of the bin packing approximation

algorithms. Both algorithms place each item into the first bin that fits, However, the First Fit Decreasing algorithm sort the items in a decreasing order before placing them. These two algorithms are the focus of this study, As the worst case performance bounds of the FF algorithm is $17/10$, and the worst case performance bounds of the FFD algorithm is $11/9$ [7] [10] [12] [3].

The FF and FFD algorithms reach near-optimal results, however, they represent a challenging complexity of $O(n^2)$ for using a simple list structure or $O(n \log n)$ for using a tree structure. Moreover, The trade-off between the utilization rate and the processing time is an important challenge to balance. Utilization is represented by the number of bins used. Furthermore, the ratio between the used and unused area of the bin is an important parameter in measuring utilization.

This paper focuses on one-dimensional bin packing algorithms, specifically the First Fit (FF) and the First Fit Decreasing (FFD) algorithm. This paper compares the FF algorithm and two variants of the FFD algorithm, one variant where the sort is sequential and the other where the sort is parallel. The main contribution of this paper is to study the effect of parallel processing on the execution time and the utilization rate of the FFD algorithm. Furthermore, The relation between the degree of parallelism represented by the number of parallel workers working concurrently, and the speed up is explored.

II. RELATED WORK

The bin packing problem has been researched in the past from different angles and approaches. This section is therefore divided by the optimization approach taken by different researchers. Section II-A discusses the traditional optimization efforts in bin packing. Also, section II-B discusses the parallel approach to the bin packing problem.

A. Traditional optimization efforts

The motivation to use the FFD in this research was the work by Johnson et al. [9]. They argued that although the worst-case

bound of the algorithm is to use 22.2% more space than the optimal solution, that the algorithm actually uses no more than 2% more spaces on average.

Zehmakan et al. [16] proposed two algorithms in the bin packing problem. The first Algorithm was a $3/2$ approximation algorithm had a similar time order to the FFD algorithm but used less space, as the FFD algorithm saves all bins during execution. The second algorithm is a modification to the FFD algorithm that decreases the time order to linear.

Balogh et al. [1] focused on the parametric case of the on-line bin packing algorithms. They offered mathematical proof that sorting the objects to be packed in a decreasing yields a lower bound of $54/47$. They also argued that if the size of the largest elements is in the interval between $8/29$ and $1/2$ then the FFD algorithm yields upper bound of $71/60$. Therefore, in this paper we generated the objects to be normally distributed, as to minimize the possibility of large objects.

B. Parallel processing optimization efforts

Varsamis et al. [15] implemented a MATLAB program to parallelize the best fit and the best fit decreasing algorithm. They aimed to reduce computation time by dividing the problem into sub-problems and applying the best fit decreasing algorithm to each sub-problem. They came to the conclusion that in order to divide the data into partitions, each partition must be representative of the whole data set to achieve the best results. They also noticed that in large data sets when the number of objects is 2^{16} and 2^{18} , the parallelization of the BFD algorithm decreased the time from 50% to 90%. Similarly, Ghosh et al. [7] implemented multiple Parallel algorithms for the bin packing problem. They demonstrated the usage of the task and data parallelism.

III. METHODOLOGY

This section discusses the implementation. Firstly, section III-A discusses the problem instances. Secondly, section III-B discusses the First Fit Algorithm and how it is implemented. Finally, the First Fit Decreasing Algorithm is discussed with its two variants, the sequential III-C and the Parallel III-D.

A. Problem instances

The problem instances in bin packing are defined by four parameters. Bin capacity (BC) is one of these parameters, which represent the maximum capacity of all the bins. The second parameter is the number of objects (n). The last two parameters are the lower (v_1) and upper (v_2) bounds, which are fractions between 0 and 1 such that the objects are between $v_1 \times BC$ and $v_2 \times BC$. Any bin packing problem instance can be represented by the tuple (BC, n, v_1, v_2) . Throughout this paper, different tuples which correspond to different data sets are used. Furthermore, all the objects are randomly generated and normally distributed to minimize the chance of having really low or really high values.

B. First Fit Algorithm

The First Fit Algorithm is an optimization algorithm for bin backing. The algorithm simply places the objects into the first bin with enough space. If there was no opened bin with enough space, a new bin is added. Figure 1 shows a pseudo-code for the algorithm.

Algorithm 1 First-Fit

```

1: for All objects  $i = 1, 2, \dots, n$  do
2:   for All bins  $j = 1, 2, \dots$  do
3:     if Object  $i$  fits in bin  $j$  then
4:       Pack object  $i$  in bin  $j$ .
5:       Break the loop and pack the next object.
6:     end if
7:   end for
8:   if Object  $i$  did not fit in any available bin then
9:     Create new bin and pack object  $i$ .
10:  end if
11: end for

```

Fig. 1: Pseudo code for first fit algorithm

C. First Fit Decreasing Algorithm (sequential)

The First Fit Decreasing Algorithm starts with sorting the objects in decreasing order. Then the algorithm continues the same as the First Fit Algorithm. In the sequential version of this algorithm, an implementation of the merge sort was employed to sort the objects. The merge sort algorithm works in two steps. The first step is a recursive function that divides a list into two halves until it can no more be divided. The second step is to merge the smaller lists into a new ordered list. Figure 2 is an Example on the merge sort. The figure shows that the array is divided until it can no longer be divided. In this example, level 4 reaches a stage at which the array can no longer be divided. At this step of the algorithm, the left most sub array can now be merged. The merge occurs in ascending order to insure that the final result is a sorted array. Merge sort was chosen because it is stable and remains consistent with a variety of problem sizes. Furthermore, the merge sort has a worst-case time complexity of $O(n \log n)$.

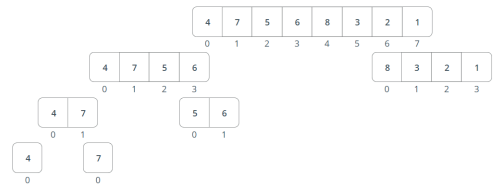


Fig. 2: An example of how merge sort works.

D. First Fit Decreasing Algorithm (Parallel)

The parallel version of the First Fit Decreasing algorithm implements a parallel merge sort. Since merge sort already uses the divide and conquer approach, it benefits from penalization. Each worker divides the objects list into two halves, this process is repeated recursively until each worker holds only one element. Then, each worker compares and merge two elements together in decreasing order until the entire array is sorted. Figure 3 Shown an example on parallel merge sort.

The figure shows that each worker divides the array into 2 halves and checks if the sub-array can be divided further. Then each worker merges two sorted sub-arrays into one array. This process repeats until the whole array is sorted. Notice that in this example the sort is ascending while the implementation is descending. Furthermore, We added a parameter to the parallel implementation of FFD algorithm that controls the level of parallelism. The level of parallelism is the maximum number of allowed workers to run concurrently.

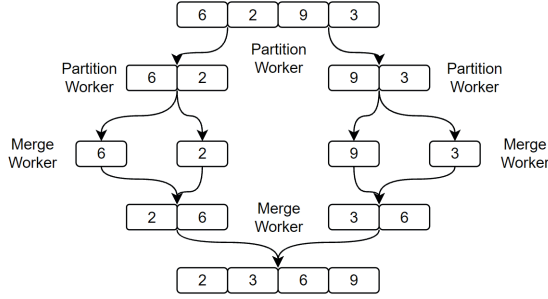


Fig. 3: An example of The Parallel merge sort.

IV. EXPERIMENT

This section discusses two experiments and the environment in which they were executed.

A. Setup environment

In order to study the effects of parallelism on the performance of the bin packing problem, We must set up a standardized environment. All the experiments performed in this paper had the following environment: Processor Intel Core i7-10750H 2.60GHz and 16 GB main memory.

B. Experiment one

In this experiment, we explore the effect of using different numbers of workers on the processing time of the FFD algorithm.

1) *Objective:* This experiment was performed to analyze some questions. Firstly, how can we parallelize the sorting step in the FFD algorithm to affect the computing time? Does it affect the utilization? Secondly, parallelization is known to cause unnecessary overhead for small problem instances, then at which point does the parallelization becomes effective?

2) *Method:* In order to answer those questions, we need to keep a few factors from affecting the experiment. Therefore, in this experiment, all the bins have bin capacity of 500, lower bound 0.2 and upper bound 0.8. The size of the problems is from 10000 to 60000. The objects are randomly generated between the lower bound and the upper bound. This experiment is focused on the parallel FFD algorithm. Each of the problem instances is solved with the FFD parallel algorithm with a different number of

number of objects	number of bins	total spaces	average waste
10000	5018	335459.37	66.85120
15000	7536	676891.59	89.82107
20000	10009	356514.4	35.61938
25000	12476	272495.59	21.84158
30000	15155	1692539.83	111.68194
35000	17570	1013370.2	57.67616
40000	20181	2195508.82	108.79088
45000	22591	1697240.31	75.12904
50000	25123	1330406.11	52.95570
55000	27593	1323925.87	47.98049
60000	30139	1704587.67	56.55753

TABLE I: The utilization rate of the FFD parallel algorithm. The average waste is the number of total unused spaces divided by the number of bins used.

concurrent workers from 2 workers to 12 workers. For each (problem size, number of workers) pair, the average time of 10 trails is measured. Indeed, the processing time is the main point that we are focusing on in this paper. However, we also tested the utilization rate of the algorithm as well. Parameters like the number of used bins, the number of unused spaces and average empty space per bin are computed.

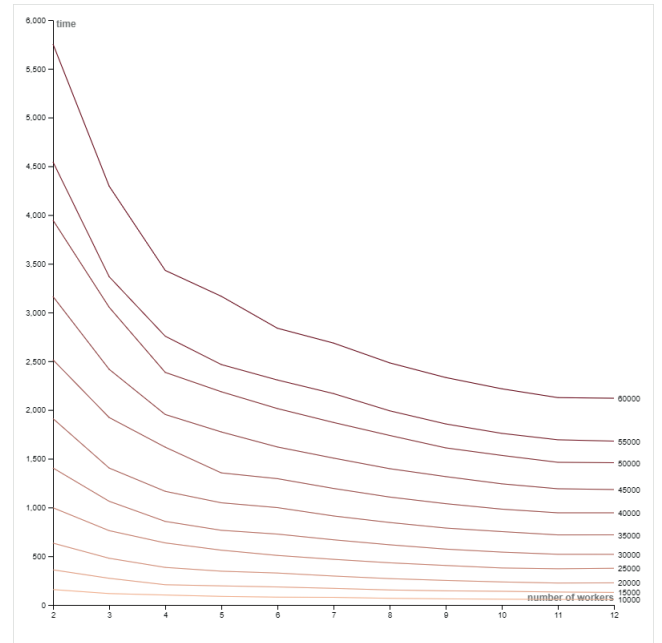


Fig. 4: The X axis represent the number of concurrent workers. The Y axis represent time in milliseconds. Each line is a problem instance with different size.

3) *Result:* Regarding the effect of parallelization on the processing time, Figure 4 shows the relation between the number of workers and the time in milliseconds. Overall, it is clear that by increasing the number of workers, the processing time decreases. Furthermore, the relation between the problem size and the speedup is also clear. All the problem sizes in the experiment have benefited from parallelization, However, the speed up differs from one instance to another. For example,

for the smallest problem size 10000, the processing time for 2,3 and 12 workers is 157.57, 116.7 and 55.139 milliseconds respectively. However, for the biggest problem size 60000, the processing time for 2,3 and 12 workers is 5752, 4295.9 and 2120.3 milliseconds respectively. Furthermore, it is safe to say that the pattern shown in figure 4 is expected to be expanded for larger problem sizes. This proves that the FFD algorithm can benefit greatly from any level of parallelization in large problem sizes.

This benefit of decreased processing time has no negative effect. Meaning that the utilization rate of the FFD is not affected. That is because the original sequential FFD algorithm also sorts the objects to be packed before processing them. Therefore, it is not surprising that the results in I are the same for both the sequential and the parallel versions on the FFD. The table describes the number of used bins, total empty spaces and the average waste per bin. The number of used bins is the main attribute for evaluating a bin packing algorithm. Table I shows that for the problem instance provided, the number of used bins are always about half that off the number of objects. Therefore, for these problem instances, the relation between the number of used bins and the problem size appears to be liner.

C. Experiment Two

1) *Objective:* In this experiment, we compared The three algorithms FF, FFDS (Sequential) and FFDP (Parallel) to find the processing time and number of used bins for each algorithm.

2) *Method:* In this experiment, several data sets were used. All the data sets follow the tuple $(500, n, 0.2, 0.8)$, where that the number of objects n are randomly generated from 100000 and 160000. This gives us a chance to focus on the size of the problem, as the differences between each data set in the number of objects needing to be packed. Each data set was fed into each of our 3 algorithms 10 times.

3) *Result:* Figure 5 is a graph representing the average time in seconds for each data set. The parallel FFD algorithm shows the best results. This proves that just by parallelizing the sorting in the FFD algorithm, we can save a lot of time. The FFDP algorithm is 21.9% faster than the FF algorithm and 4.73% faster than the FFDS algorithm. The number of workers in this experiment was 12, due to the set up environment mentioned above. However, if using 12 workers achieved 4.73% speed up, then by the relation proved in the previous experiment, the potential of this approach seems promising.

Regarding the utilization rate of every algorithm, figure II shows the relation between the size of the data set as a number of objects and the utilization as the number of used bins. The FFDP and FFDS algorithms have the same number of bins as the only difference between them is the sorting

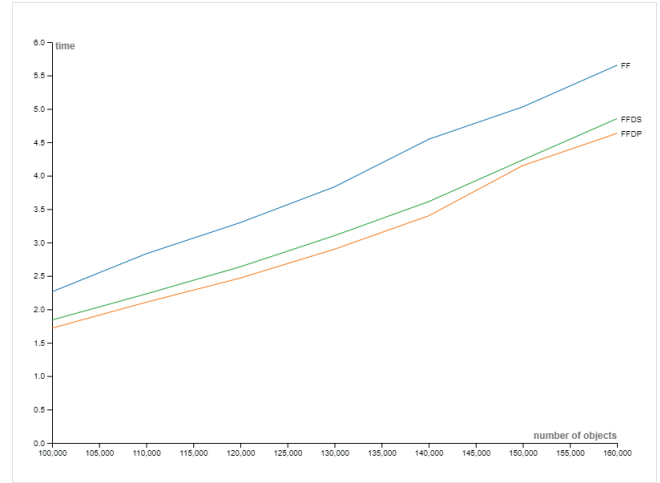


Fig. 5: A graph representing the Average time in seconds for each problem instance. Each instance has a different number of objects. The parallel FFD algorithm shows the best results.

type	used bins	total spaces	average waste	problem size
FF	50775	389331.76	7.66778454	100000
FFDS	50126	64831.76	1.293375893	100000
FFDP	50126	64831.76	1.293375893	100000
FF	56002	473080.16	8.447558301	110000
FFDS	55206	75080.16	1.36	110000
FFDP	55206	75080.16	1.36	110000
FF	60915	448929.99	7.369777395	120000
FFDS	60143	62929.99	1.046339391	120000
FFDP	60143	62929.99	1.046339391	120000
FF	65810	442565.92	6.724903814	130000
FFDS	64982	28565.92	0.439597427	130000
FFDP	64982	28565.92	0.439597427	130000
FF	71035	528659.9	7.442245372	140000
FFDS	70086	54159.9	0.772763462	140000
FFDP	70086	54159.9	0.772763462	140000
FF	76123	549749.39	7.221856601	150000
FFDS	75164	70249.39	0.934614842	150000
FFDP	75164	70249.39	0.934614842	150000
FF	81121	558161.48	6.880604036	160000
FFDS	80132	63661.48	0.794457645	160000
FFDP	80132	63661.48	0.794457645	160000

TABLE II: This table shows the utilization of the three algorithms for different problem sizes. The average waste is the total number of empty spaces divided by the number of used bins.

process. Moreover, the FFD algorithms use less bins than the FF algorithm. The average waste results show how the average bin is utilized and how much of it is wasted with empty space. Also, the FFD utilizes 99.84% of the used bins when the problem size is 60000. Meaning that only 0.16% of each bin is empty.

V. CONCLUSION AND FUTURE WORK

In conclusion, this paper proposed a parallel approach to the FFD algorithm. We proved that just by parallelizing the sorting in the First Fit Decreasing algorithm, we can decrease the computation time by 4.73% without affecting the utilization rate. Future work may include parallelizing a two-dimensional

bin packing algorithm. Furthermore, the proposed algorithm can be optimized to use less space, as the FFD algorithm saves all bins in memory.

REFERENCES

- [1] J. Balogh, J. Békési, and G. Galambos, "New lower bounds for certain classes of bin packing algorithms," *Theoretical Computer Science*, vol. 440-441, pp. 1–13, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0304397512003611>
- [2] E. G. Coffman, M. R. Garey, and D. S. Johnson, "An application of bin-packing to multiprocessor scheduling," *SIAM J. Comput.*, vol. 7, pp. 1–17, 1978.
- [3] G. Dósa and J. Sgall, "First Fit bin packing: A tight analysis," in *30th International Symposium on Theoretical Aspects of Computer Science (STACS 2013)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), N. Portier and T. Wilke, Eds., vol. 20. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2013, pp. 538–549. [Online]. Available: <http://drops.dagstuhl.de/opus/volltexte/2013/3963>
- [4] L. Epstein and A. Levin, "Asymptotic fully polynomial approximation schemes for variants of open-end bin packing," *Information processing letters*, vol. 109, no. 1, pp. 32–37, 2008.
- [5] E. Falkenauer, A. Delchambre *et al.*, "A genetic algorithm for bin packing and line balancing," in *ICRA*, 1992, pp. 1186–1192.
- [6] M. R. Garey, R. L. Graham, and J. D. Ullman, "Worst-case analysis of memory allocation algorithms," in *Proceedings of the fourth annual ACM symposium on Theory of computing*, 1972, pp. 143–150.
- [7] S. Ghosh and A. H. Gebremedhin, "Parallelization of bin packing on multicore systems," in *2016 IEEE 23rd International Conference on High Performance Computing (HiPC)*, 2016, pp. 311–320.
- [8] D. S. Hochbaum, Ed., *Approximation Algorithms for NP-Hard Problems*. USA: PWS Publishing Co., 1996.
- [9] D. S. Johnson, "Near-optimal bin packing algorithms," 1973.
- [10] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham, "Worst-case performance bounds for simple one-dimensional packing algorithms," *SIAM Journal on computing*, vol. 3, no. 4, pp. 299–325, 1974.
- [11] W. Leinberger, G. Karypis, and V. Kumar, "Multi-capacity bin packing algorithms with applications to job scheduling under multiple constraints," in *Proceedings of the 1999 International Conference on Parallel Processing*. IEEE, 1999, pp. 404–412.
- [12] E. C. man Jr, M. Garey, and D. Johnson, "Approximation algorithms for bin packing: A survey," *Approximation algorithms for NP-hard problems*, pp. 46–93, 1996.
- [13] A. Martinez-Sykora, R. Alvarez-Valdés, J. A. Bennell, R. Ruiz, and J. M. Tamarit, "Matheuristics for the irregular bin packing problem with free rotations," *European Journal of Operational Research*, vol. 258, no. 2, pp. 440–455, 2017.
- [14] A. Ntanjana, "Two and three-dimensional bin packing problems: an efficient implementation of evolutionary algorithms," Ph.D. dissertation, 2018.
- [15] D. Varsamis and F. Chanlioglou, "A parallel approach of best fit decreasing algorithm [j]," *Wseas Transactions on Computers*, vol. 17, no. 9, pp. 79–85, 2018.
- [16] A. N. Zehmakan, "Bin packing problem: Two approximation algorithms," *arXiv preprint arXiv:1508.01376*, 2015.